Chuleta Docker

Imagen Python (Flask) e instrucciones del Dockerfile

Se usa el usuario **jose93ln** de Docker Hub. Si el tuyo es otro, cámbialo en todos los comandos.

Índice

1. Imagen de una app Python (Flask) 2
2. Construir y ejecutar 3
3. Gestionar el contenedor 3
4. Subir a Docker Hub 3
5. Errores típicos 3
6. Instrucciones del Dockerfile que hay que dominar 3
7. Ejemplo completo con las 7 instrucciones 5
8. Resumen rápido (copiar y pegar) 6

1. Imagen de una app Python (Flask)

Estructura del proyecto

```
exPython/
  app.py
  requirements.txt
  Dockerfile
```

app.py

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def inicio():
    return "Hola desde Docker"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Importante: `host="0.0.0.0"` hace que Flask escuche en todas las interfaces. Con `127.0.0.1` solo se podría acceder desde dentro del contenedor y el navegador no cargaría nada.

requirements.txt

```
flask
```

Lista de librerías que pip instalará. Una por línea.

Dockerfile

```
FROM python:3.12
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

Usa el tag de Python que te indique el profesor. Si pones uno que no existe en Docker Hub, el build falla con *manifest not found*: comprueba el tag en hub.docker.com.

Instrucción	Qué hace
`FROM`	Imagen base (Linux + Python).
`WORKDIR /app`	Carpeta de trabajo dentro de la imagen. Si no existe, se crea.
`COPY requirements.txt .`	Copia el archivo de tu PC a /app (el punto es la carpeta actual del contenedor, o sea /app).
`RUN pip install -r requirements.txt`	Se ejecuta **al construir** la imagen e instala las librerías.
`COPY app.py .`	Copia el código a /app.
`EXPOSE 5000`	Solo **documenta** el puerto de la app, no lo abre. Quien lo abre es -p.
`CMD [...]`	Comando que se ejecuta **al arrancar** el contenedor.

RUN vs CMD: `RUN` se ejecuta en `docker build` (una vez, al crear la imagen). `CMD` se ejecuta en `docker run` (cada vez que arranca un contenedor).

Truco de caché: se copia `requirements.txt` antes que `app.py` porque Docker guarda en caché cada capa. Si solo cambias app.py, reutiliza la capa del pip install y el build es mucho más rápido.

2. Construir y ejecutar

```
docker build -t jose93ln/app-py:1.0 .
docker run -d --name mi-app -p 5000:5000 jose93ln/app-py:1.0
curl http://localhost:5000
```

- `-t usuario/nombre:tag`: nombre de la imagen; debe empezar por tu usuario de Docker Hub. El **punto final** es el contexto (carpeta actual).
- `-d`: segundo plano. `--name mi-app`: nombre del contenedor.
- `-p host:contenedor`: publica el puerto. `-p 8000:5000` significa que entras por localhost:8000.
- `curl` o el navegador en http://localhost:5000 para probar.

3. Gestionar el contenedor

Comando	Para qué sirve
`docker ps`	Contenedores en ejecución.
`docker ps -a`	Todos, también los parados.
`docker logs mi-app`	Ver los logs.
`docker logs -f mi-app`	Logs en directo (Ctrl+C para salir).
`docker stop mi-app`	Parar.
`docker start mi-app`	Arrancar de nuevo.
`docker rm mi-app`	Borrar (tiene que estar parado).
`docker rm -f mi-app`	Borrar aunque esté en marcha.

4. Subir a Docker Hub

```
docker login -u jose93ln
docker push jose93ln/app-py:1.0
```

Crear el repositorio a mano en Docker Hub es opcional, porque se crea solo en el primer push. Si da *insufficient_scope*, créalo a mano con el mismo nombre que la imagen (app-py).

5. Errores típicos

Problema	Causa / solución
No carga en el navegador	Falta host="0.0.0.0" en app.py o falta -p.
Cambié app.py y no se nota	Hay que volver a hacer docker build y recrear el contenedor.
push da error de acceso	La imagen no empieza por tu usuario o falta docker login.
Nombre de contenedor en uso	docker stop mi-app && docker rm mi-app
Puerto ocupado	Cambia el puerto del host: -p 8000:5000.

6. Instrucciones del Dockerfile que hay que dominar

Resumen: **FROM, LABEL, ENV, WORKDIR, COPY, RUN, CMD**.

FROM: imagen base

Es **siempre la primera instrucción**. Indica de qué imagen partes.

```
FROM nginx:1.27-alpine
FROM python:3.12
FROM ubuntu:22.04
```

- Formato `imagen:tag`. Sin tag usa `latest` (mala práctica; mejor fijar versión).
- `alpine` en el tag es una versión ligera de Linux (usa apk). Sin él suele ser Debian (usa apt-get).

LABEL: metadatos

Añade información a la imagen en formato clave=valor. No cambia el funcionamiento, solo documenta.

```
LABEL maintainer="jose93ln"
LABEL version="1.0"
LABEL description="Servidor web con nginx"

# Varias en una sola línea (menos capas):
LABEL maintainer="jose93ln" version="1.0" description="Servidor web con nginx"
```

Para verlas:

```
docker inspect --format '{{json .Config.Labels}}' jose93ln/web-nginx:1.0
```

ENV: variables de entorno

Define variables que existen **durante el build y también con el contenedor en ejecución**.

```
ENV APP_ENV=produccion
ENV PUERTO=5000
ENV APP_ENV=produccion PUERTO=5000
```

Se pueden usar en instrucciones posteriores con $VARIABLE, y desde la app (en Python: os.environ["APP_ENV"]).

```
ENV CARPETA=/app
WORKDIR $CARPETA
```

Comprobar y cambiar al arrancar:

```
docker exec mi-app env                                    # ver las variables
docker run -d -e APP_ENV=desarrollo jose93ln/app-py:1.0     # sobrescribir con -e
```

WORKDIR: directorio de trabajo

Cambia el directorio donde se ejecutan las instrucciones siguientes (RUN, COPY, CMD...). Si la carpeta no existe, la crea.

```
WORKDIR /app
COPY app.py .        # se copia a /app/app.py
RUN ls               # se ejecuta dentro de /app
```

Es como un `cd` permanente para el resto del Dockerfile. Mejor `WORKDIR` que `RUN cd /app`, porque ese cd solo dura una línea.

COPY: copiar archivos a la imagen

Formato: `COPY origen destino`. El **origen** es relativo al contexto de build (carpeta donde haces docker build). El **destino** es una ruta dentro de la imagen.

```
COPY index.html /usr/share/nginx/html/index.html   # un archivo con ruta completa
COPY app.py .                                      # al WORKDIR actual
COPY . .                                           # toda la carpeta del proyecto
COPY carpeta/ /destino/                            # una carpeta entera
```

No se pueden copiar archivos fuera de la carpeta del build (`COPY ../algo` no funciona).

RUN: ejecutar comandos al construir

Se ejecuta **durante docker build** y el resultado queda guardado en la imagen. Se usa para instalar paquetes, crear carpetas, etc.

```
RUN apt-get update && apt-get install -y curl      # Debian
RUN apk add --no-cache nano                        # Alpine
RUN pip install -r requirements.txt                # Python
RUN mkdir /datos
```

- `&&` encadena comandos (el segundo solo se ejecuta si el primero tuvo éxito). En un solo RUN genera menos capas.
- `-y` en apt-get install evita la pregunta de confirmación, que en un build no puedes contestar.

CMD: comando por defecto al arrancar

Se ejecuta **cuando haces docker run**, no al construir. Solo cuenta **un CMD por Dockerfile** (si hay varios, vale el último).

```
CMD ["python", "app.py"]                    # forma exec (recomendada)
CMD python app.py                           # forma shell
CMD ["nginx", "-g", "daemon off;"]
```

- **Forma exec** (corchetes y comillas dobles): ejecuta el programa directamente. Es la recomendada.
- **Forma shell** (sin corchetes): se ejecuta a través de /bin/sh -c.
- Se puede sobrescribir al ejecutar: docker run jose93ln/app-py:1.0 python otro.py.

Comparativa RUN vs CMD

	RUN	CMD
Cuándo se ejecuta	En docker build	En docker run
Cuántos puede haber	Todos los que quieras	Solo uno (vale el último)
Para qué	Instalar y preparar la imagen	Arrancar la aplicación

7. Ejemplo completo con las 7 instrucciones

app.py

```
import os
from flask import Flask

app = Flask(__name__)

@app.route("/")
def inicio():
    entorno = os.environ.get("APP_ENV", "sin definir")
    return f"Hola desde Docker. Entorno: {entorno}"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

requirements.txt

```
flask
```

Dockerfile

```
FROM python:3.12-slim
LABEL maintainer="jose93ln" version="1.0" description="App Flask de ejemplo"
ENV APP_ENV=produccion
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
CMD ["python", "app.py"]
```

Qué hace cada línea, en orden:

- `FROM`: parte de una imagen con Python 3.12 (versión slim, más ligera).
- `LABEL`: anota autor, versión y descripción.
- `ENV`: crea la variable APP_ENV, que la app lee con os.environ.
- `WORKDIR`: trabaja dentro de /app.
- `COPY requirements.txt .`: copia la lista de librerías.
- `RUN pip install ...`: instala Flask al construir (--no-cache-dir reduce el tamaño).
- `COPY app.py .`: copia el código.
- `CMD`: arranca la aplicación cuando se ejecuta el contenedor.

Probarlo

```
docker build -t jose93ln/app-py:1.0 .
docker run -d --name mi-app -p 5000:5000 jose93ln/app-py:1.0
curl http://localhost:5000        # "Hola desde Docker. Entorno: produccion"

# Cambiar la variable sin tocar el Dockerfile:
docker rm -f mi-app
docker run -d --name mi-app -p 5000:5000 -e APP_ENV=desarrollo jose93ln/app-py:1.0
curl http://localhost:5000        # "... Entorno: desarrollo"

# Comprobar el LABEL y el ENV:
docker inspect --format '{{json .Config.Labels}}' jose93ln/app-py:1.0
docker exec mi-app env
```

Orden recomendado: `FROM`, `LABEL`, `ENV`, `WORKDIR`, `COPY` de dependencias, `RUN`, `COPY` del código, `CMD`. Lo que menos cambia va arriba y lo que más cambia (tu código) abajo, para aprovechar la caché.

8. Resumen rápido (copiar y pegar)

```
cd ~/exPython
docker build -t jose93ln/app-py:1.0 .
docker run -d --name mi-app -p 5000:5000 jose93ln/app-py:1.0
curl http://localhost:5000
docker logs mi-app
docker login -u jose93ln
docker push jose93ln/app-py:1.0
```